

MATRICES Y CICLOS ANIDADOS

Profesor: Pablo J Piccoli Bornia

Auxiliares: Daniela Longas - M Victoria Tolosa

MATRICES Y ESTRUCTURAS BI- DIMENSIONALES

Estructuras Bi-dimensionales

A medida que los problemas se vuelven complejos, incorporamos estructuras avanzadas para manejar la información, datos u otros elementos. Las **listas** y los **diccionarios** son algunos ejemplos, pero no los únicos.

Una estructura muy frecuente, en particular en algoritmos que construyen sobre herramientas matemáticas, son las **matrices**. Una matriz $A \in \mathbb{R}^{m \times n}$ tiene m filas y n columnas, donde a_{ij} representa el elemento de la i -ésima fila y j -ésima columna. Un ejemplo de una matriz $A \in \mathbb{R}^{4 \times 3}$.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix}$$

Podemos representar una matriz con las estructuras que conocemos?

Matrices y lista de listas

Una posible representación para una **matriz** es mediante lista de listas. Es la forma natural de manejar una **estructura bi-dimensional** en gran parte de los contextos de aplicación.

Hay múltiples formas de representar una matriz mediante listas de listas. Dos casos clásicos:

- Representación por filas. La lista principal representa una colección de *filas*, y el *i*-ésimo elemento de esta lista es **otra** lista con los *elementos de la fila i*.
- Representación por columnas. La lista principal representa una colección de *columnas*, y el *j*-ésimo elemento de esta lista es **otra** lista con los *elementos de la columna j*.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix}$$

```
>>> A = [[1,2,3],  
          [4,5,6],  
          [7,8,9],  
          [10,11,12]]
```

```
>>> A = [[1,4,7,10],  
          [2,5,8,11],  
          [3,6,9,12]]
```

Matrices y Estructuras Bi-dimensionales

Ejercicio: Queremos acceder al elemento a32. Cuál es índice de la fila y cuál el índice de la columna?

Ejercicio: Queremos acceder al elemento a32. Cuál es índice de la fila y cuál el índice de la columna? Recordar que las listas indexan de 0 a $n - 1$. El elemento a32 corresponde a la fila 2 y columna 1 de la matriz en Python.

- Representación por filas.

```
>>> A = [[1,2,3],[4,5,6],[7,8,9],[10,11,12]]
```

```
>>> A[2] # Que nos devuelve ?
```

```
[7,8,9] # Es la lista de la posicion 2 .
```

```
>>> A[2][1] # Que nos devuelve ?
```

Matrices y Estructuras Bi-dimensionales

- Representación por columnas.

```
>>> A = [[1,4,7,10],[2,5,8,11],[3,6,9,12]]
```

```
>>> A[1] # Que nos devuelve ?
```

```
[2,5,8,11] # Es la lista de la posicion 2 .
```

```
>>> A[1][2] # Que nos devuelve ?
```

Al tener una lista de lista, podemos acceder a los elementos mediante la aplicación secuencial del operador []. Es fundamental tener presente la representación adoptada para almacenar la información.

Matrices y Listas de Listas

Supongamos que tenemos una matriz de float A 2 Rmn **representada por filas** mediante **listas de listas**. Buscamos implementar una función *suma_elementos* que retorne la suma de todos los elementos de la matriz.

A modo de ejemplo, para la matriz

$$A = \begin{bmatrix} [1, 2, 3], \\ [4, 5, 6], \\ [7, 8, 9] \end{bmatrix}$$

el resultado es 78.

Recorriendo una matriz: por filas

Idea general del algoritmo:

- Vamos a tener que recorrer eventualmente todos los elementos de la matriz.
- El resultado va a ser un número.
- Cuando recorrimos todos los elementos, retornamos el valor.
- La función suma lista recibe la cantidad de filas (m) y columnas (n).
- Asumimos que está bien construída.

```
def suma_elementos(A, m, n) :  
    ret = 0.0 # acumulador inicializado en 0  
    i = 0 # variable que indica en que fila estamos  
    # Recorremos las filas. Terminamos cuando i = m  
    while i < m:  
        j = 0 # variable que indica en que columna estamos.  
        # Recorremos las columnas de la fila i. Terminamos con j = n  
        while j < n :  
            ret = ret + A[i][j] # Actualizamos el valor de la suma  
            j = j + 1  
        i = i + 1 # pasamos a la siguiente fila.  
    return ret
```

Recorriendo una matriz: por filas

```
def suma_elementos(A, m, n) :  
    ret = 0.0 # acumulador inicializado en 0  
    i = 0 # variable que indica en que fila estamos  
  
    # Recorremos las filas. Terminamos cuando i = m  
    while i < m:  
        j = 0 # variable que indica en que columna estamos.  
        # Recorremos las columnas de la fila i. Terminamos con j = n  
        while j < n :  
            ret = ret + A[i][j] # Actualizamos el valor de la suma  
            j = j + 1  
  
        i = i + 1 # pasamos a la siguiente fila.  
    return ret
```

Cómo podríamos hacer el código más modularizado usando una función auxiliar?

Recorriendo una matriz: por filas

El siguiente extracto del código está sumando los elementos de una fila.

```
j = 0 # variable que indica en que columna estamos.  
# Recorremos las columnas de la fila i. Terminamos con j = n  
while j < n :  
    ret = ret + A[i][j] # Actualizamos el valor de la suma.  
    j = j + 1
```

Entonces, podemos usar una función auxiliar que dada A, la cantidad de columnas n, el índice de una fila i (con $i < m$) retorne un float sumando los elementos de la fila.

```
def sumar_elementos_fila_i(A,n,i):  
    ret = 0 # acumulador para suma de la fila.  
    j = 0 # variable que indica en que columna estamos .  
    # Recorremos las columnas de la fila i. Terminamos con j =  
    n  
    while j < n :  
        ret = ret + A[i][j] # Actualizamos el valor de la suma  
        j = j + 1  
    return ret
```

Recorriendo una matriz: modularizado

```
def sumar_elementos_fila_i(A,n,i):  
    ret = 0 # acumulador para suma de la fila.  
    j = 0 # variable que indica en que columna estamos.  
    # Recorremos las columnas de la fila i. Terminamos con j =  
    n  
    while j < n :  
        ret = ret + A[i][j] # Actualizamos el valor de la suma  
        j = j + 1  
    return ret  
def suma_elementos(A,m,n):  
    ret = 0.0 # acumulador inicializado en 0  
    i = 0 # variable que indica en que fila estamos.  
  
    # Recorremos las filas. Terminamos cuando i = m  
    while i < m:  
        ret = ret + sumar_elementos_fila_i (A,n,i)  
        i = i + 1 # pasamos a la siguiente fila  
    return ret
```

Ejercicio

Supongamos que tenemos una matriz $A \in \mathbb{R}^{m \times n}$ **representada por filas** mediante **listas de listas**. Buscamos implementar:

1. Una función *suma_por_filas* que recibe la matriz A y retorna una lista de m posiciones, donde la i -ésima posición contiene la suma de los elementos de la fila i . A modo de ejemplo, para la matriz A usada como ejemplo debe retornar la lista $[6, 15, 24, 33]$.
2. Una función *suma_por_columnas* que recibe la matriz A y retorna una lista de n posiciones, donde la j -ésima posición contiene la suma de los elementos de la fila j . A modo de ejemplo, para la matriz A usada como ejemplo debe retornar la lista $[22, 26, 30]$.

Ejercicio

Idea general del algoritmo

- Vamos a tener que recorrer eventualmente todos los elementos de la matriz.
- El resultado va a ser una lista con m elementos. La posición i se corresponde con la suma de la fila i .
- Podemos calcular de a una fila por vez. *Iteramos todas las filas 1 a m (0 a $m - 1$).*
- Cuando estamos analizando la fila i , entonces queremos sumar los elementos $a_{i1}; a_{i2}; \dots; a_{in}$.
- Cuando recorrimos todas las filas, retornamos la lista.

```
def suma_por_filas (A, m, n):  
    ret = [0] * m # lista de m posiciones inicializada en 0  
    for i in range (m): # Recorremos las filas.  
        for j in range (n): # Recorremos las columnas de la fila i  
            ret[i] = ret[i] + A[i][j] # Actualizamos valor de la suma  
    return ret
```

CICLOS ANIDADOS MÁS ALLÁ DE MATRICES

Detectando errores de tipeo

Es muy común que al tipear un texto en la computadora o el teléfono muchas veces nos equivoquemos al presionar las teclas. Muchos de estos errores pueden ser identificados fácilmente y corregidos mediante algún programa que analice lo que escribimos y proponga una sugerencia. Vamos a considerar solamente un tipo simple y común de *typos*, y pensar cómo resolverlo con las herramientas que tenemos.

Un error muy común es escribir una palabra y equivocarnos en alguna de las letras, presionando alguna cercana alrededor. A modo de ejemplo, en un teclado *qwerty*, al querer tipear la palabra *casa*

- podemos confundir la *s* con la *d* y tipear *cada*;
- o confundir la *a* con la *s* y escribir *csss*;
- también podría suceder que por la velocidad de escritura se invierta el orden en el que se presionan las letras, en lugar de escribir *casa* obtenemos *caas*.

Una manera muy elemental de identificar estos casos consiste en definir una medida de distancia entre dos palabras. A fines prácticos, vamos a usar una definición ad-hoc para ejercitar.

Detectando errores de tipeo

Una manera muy elemental de identificar estos casos consiste en definir una medida de distancia entre dos palabras. A fines prácticos, vamos a usar una definición ad-hoc para ejercitar.

Decimos que dos palabras son **similares** si:

1. tienen la misma cantidad de caracteres, y
2. la cantidad de posiciones con caracteres distintos menor o igual que 2.

Se pide:

1. Completar la función `son_similares`.
2. Completar la función `pares_similares`, que dada una lista de palabras retorna todos los pares de palabras distintas similares. A modo de ejemplo, para

`l = [ ' casa ', ' caas ', ' cada ', ' csad ' ]`

debería retornar (independientemente del orden)

`[ ( ' casa ', ' caas ' ), ( ' casa ', ' cada ' ), ( ' caas ', ' cada ' ), ( ' caas ', ' csad ' ) ]`